# DATA STRUCTURES

LECTURE 1

# AGENDA

1. Definition of Data structure
2. Abstract data type
3. Primitive data type
4. Non primitive data type
5. Linear DS vs  non linear DS
6. One dimension Array
7. Multi-dimension Array
8. Template

# 1. 1. DEFINITION OF DATA STRUCTURE

- A **Data Structure** is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently. It defines the relationship between data elements and the operations that can be performed on them.
- **Examples:** Arrays, Linked Lists, Stacks, Queues, Trees, Graphs, etc.

```
Array Example (Linear Structure):
Index: 0       1       2
Data:  Alice   Bob     Carol

Linked List Example (Non-Linear Structure):
[Alice] -> [Bob] -> [Carol] -> NULL

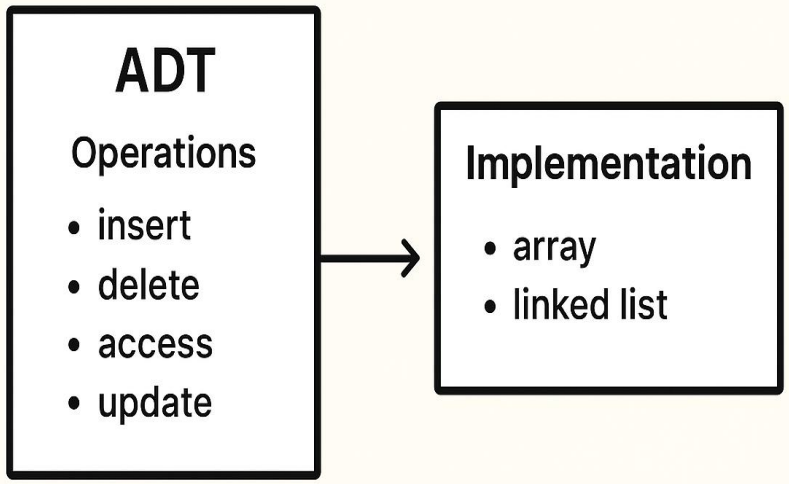
Tree Example (Hierarchical Structure):
        Alice
       /     \
     Bob     Carol
```

# 1.2. ABSTRACT DATA TYPE (ADT)

- An **Abstract Data Type (ADT)** is a **mathematical model** for a certain class of data structures that **specifies the behavior** (operations) of the data type **without specifying how it is implemented**.
- It focuses on **what operations are to be performed** rather than **how these operations are implemented**.
- ADT provides a **logical description** of the data and the operations on it.
- Examples: **Stack, Queue, List, Deque, Priority Queue, Set**

# 1.2. ABSTRACT DATA TYPE (ADT) CONT...

- ADT is a **conceptual model**; it hides the implementation details.
- Operations include:
    - Insertion
    - Deletion
    - Access/Retrieval
    - Update/Modification
- Implementation can vary:
    - Using arrays
    - Using linked lists
    - Using other data structures



# COMMON EXAMPLES OF ADTS(STACK )

- **Definition**: A stack is a collection where elements are added and removed in a **Last In, First Out (LIFO)** order.
- **Operations**:

1. `push(x)` → Add element `x` to the top.
2. `pop()` → Remove and return the top element.
3. `peek()` → Return the top element without removing it.
4. `isEmpty()` → Check if the stack is empty.

- **Implementation**: Can be done using an array or linked list.

# COMMON EXAMPLES OF ADTS(QUEUE)

- **Definition**: A queue is a collection where elements are added at the **rear** and removed from the **front** (**First In, First Out - FIFO**).
- **Operations**:

- Example :People waiting in a line ⟶ first person to come is first person served.

1. `enqueue(x)` → Add element `x` at the rear.
2. `dequeue()` → Remove and return element from the front.
3. `front()` → Get the front element without removing it.
4. `isEmpty()` → Check if the queue is empty.

# 1.3. PRIMITIVE DATA TYPES

- Primitive data types are the **basic building blocks** provided by a programming language. They are **simple** and **predefined**.
- **Characteristics:**
  - Stores **single values**.
  - **Efficient** in memory usage.
  - Predefined by the language.

**Examples:**

| Data Type | Description | Example |
|---|---|---|
| int | Stores integers | `int age = 25;` |
| float | Stores decimal numbers | `float price = 19.99;` |
| char | Stores a single character | `char grade = 'A';` |
| boolean | Stores true/false | `boolean isActive = true;` |
| double | Stores larger decimal numbers | `double pi = 3.14159;` |

# 1.4. NON-PRIMITIVE DATA TYPES

- Non-primitive data types are **more complex**, usually created by the programmer or provided by libraries. They can store **multiple values** or more **complex data structures**.
- **Characteristics:**
    - Not predefined (except some like String in Java).
    - Can store **collections of values**.
    - Can be **manipulated with methods/functions**.
    - Typically **reference types** (they store memory addresses instead of actual values).

## Examples:

| Data Type | Description | Example |
| --- | --- | --- |
| String | Stores text | `String name = "Ahmed";` |
| Arrays | Stores multiple values of same type | `int[] numbers = {1,2,3};` |
| Classes/Objects | Custom data types | `Car myCar = new Car();` |
| Lists (Java)/ArrayList | Dynamic collection of elements | `ArrayList<Integer> list = new ArrayList<>();` |

| Feature | Primitive Data Type | Non-Primitive Data Type |
| --- | --- | --- |
| Definition | Basic, predefined | Complex, user-defined or library-based |
| Size | Fixed (depends on type) | Dynamic / depends on content |
| Memory | Stored directly | Stored as reference |
| Examples | int, float, char, boolean | String, Array, Object |
| Methods | No built-in methods | Can have built-in methods |

# 1.5. LINEAR DATA STRUCTURES (LDS)

- A linear data structure is a structure in which elements are **arranged sequentially**, and each element is **connected to its previous and next element** (except the first and last).
- **Characteristics:**
  - Elements are stored **in order**.
  - **Single level** organization.
  - Traversal is **simple** (one after another).
  - Memory can be **contiguous** (like arrays) or **linked** (like linked lists).

## Examples:

| Structure | Description | Example |
| --- | --- | --- |
| Array | Collection of elements of the same type | `int arr[] = {1,2,3,4};` |
| Linked List | Elements linked using pointers | `Node1 -> Node2 -> Node3` |
| Stack | LIFO (Last In First Out) | Push/pop operations |
| Queue | FIFO (First In First Out) | Enqueue/dequeue operations |

# 1.5. NON-LINEAR DATA STRUCTURES (NLDS)

- A non-linear data structure is a structure in which elements are **not arranged sequentially**. **Each element can connect to multiple elements**, forming a **hierarchy or network**.
- **Characteristics:**
    - Elements are **not in order**.
    - **Multi-level** organization.
    - Traversal requires **more complex algorithms** (like recursion, DFS, BFS).
    - Memory is usually **dynamic**.

## Examples:

| Structure | Description | Example |
| --- | --- | --- |
| Tree | Hierarchical structure (root → children) | Binary Tree, Binary Search Tree |
| Graph | Network of nodes with edges | Social networks, Google Maps connections |

## Key Differences:

| Feature | Linear DS | Non-Linear DS |
|---|---|---|
| Arrangement | Sequential | Hierarchical / Network |
| Levels | Single level | Multiple levels |
| Traversal | Easy, one by one | Complex, needs recursion or algorithms |
| Memory usage | Contiguous or linked | Usually dynamic |
| Examples | Array, Linked List, Stack, Queue | Tree, Graph |

# 1.6.ARRAY

- An **array** is a **collection of elements of the same data type** stored in **contiguous memory locations**.
- All elements are accessed using an **index**.
- Indexing **starts from 0** in C++.
- Syntax:

```
data_type array_name[size];
```

- Example :

```
int numbers[5]; // Declares an array of 5 integers
```

# 1.6.ARRAY

- Initializing an Array:

You can assign values at the time of declaration:

```cpp
int numbers[5] = {10, 20, 30, 40, 50};
```

Or assign values later:

```cpp
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
```

# 1.6.ARRAY

- Accessing Array Elements

```
First element: 10
Third element: 30
Modified second element: 25
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    cout << "First element: " << numbers[0] << endl; // 10
    cout << "Third element: " << numbers[2] << endl; // 30

    numbers[1] = 25; // Modify second element
    cout << "Modified second element: " << numbers[1] << endl; // 25

    return 0;
}
```

# ARRAY

- Traversing an Array

```
Array elements: 10 20 30 40 50
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    cout << "Array elements: ";
    for(int i = 0; i < 5; i++) {
        cout << numbers[i] << " ";
    }
    cout << endl;

    return 0;
}
```

# 1.7.MULTIDIMENSIONAL ARRAYS

# 1.7.1. TWO-DIMENSIONAL ARRAYS

# 1.7.1. TWO-DIMENSIONAL ARRAYS

- Can define one array for multiple sets of data
- Like a table in a spreadsheet
- Use two size declarators in definition:

```
const int ROWS = 4, COLS = 3;
int exams[ROWS][COLS];
```

- First declarator is number of rows; second is number of columns

# 1.7.1. TWO-DIMENSIONAL ARRAY REPRESENTATION

```
const int ROWS = 4, COLS = 3; int exams[ROWS][COLS];
```

- Use two subscripts to access element:

```
exams[2][2] = 86;
```

columns

rows

| | | |
|---|---|---|
| exams[0][0] | exams[0][1] | exams[0][2] |
| exams[1][0] | exams[1][1] | exams[1][2] |
| exams[2][0] | exams[2][1] | exams[2][2] |
| exams[3][0] | exams[3][1] | exams[3][2] |

```
1  // This program demonstrates a two-dimensional array.
2  #include <iostream>
3  #include <iomanip>
4  using namespace std;
5
6  int main()
7  {
8     const int NUM_DIVS = 3;           // Number of divisions
9     const int NUM_QTRS = 4;           // Number of quarters
10    double sales[NUM_DIVS][NUM_QTRS]; // Array with 3 rows and 4 columns.
11    double totalSales = 0;            // To hold the total sales.
12    int div, qtr;                     // Loop counters.
13
14    cout << "This program will calculate the total sales of\n";
15    cout << "all the company's divisions.\n";
16    cout << "Enter the following sales information:\n\n";
17
```

*(program continues)*

```
18      // Nested loops to fill the array with quarterly
19      // sales figures for each division.
20      for (div = 0; div < NUM_DIVS; div++)
21      {
22         for (qtr = 0; qtr < NUM_QTRS; qtr++)
23         {
24            cout << "Division " << (div + 1);
25            cout << ", Quarter " << (qtr + 1) << ": $";
26            cin >> sales[div][qtr];
27         }
28         cout << endl; // Print blank line.
29      }
30
31      // Nested loops used to add all the elements.
32      for (div = 0; div < NUM_DIVS; div++)
33      {
34         for (qtr = 0; qtr < NUM_QTRS; qtr++)
35            totalSales += sales[div][qtr];
36      }
37
38      cout << fixed << showpoint << setprecision(2);
39      cout << "The total sales for the company are: $";
40      cout << totalSales << endl;
41      return 0;
42   }
```

**Program Output with Example Input Shown in Bold**

```
This program will calculate the total sales of
all the company's divisions.
Enter the following sales data:

Division 1, Quarter 1: $31569.45 [Enter]
Division 1, Quarter 2: $29654.23 [Enter]
Division 1, Quarter 3: $32982.54 [Enter]
Division 1, Quarter 4: $39651.21 [Enter]

Division 2, Quarter 1: $56321.02 [Enter]
Division 2, Quarter 2: $54128.63 [Enter]
Division 2, Quarter 3: $41235.85 [Enter]
Division 2, Quarter 4: $54652.33 [Enter]

Division 3, Quarter 1: $29654.35 [Enter]
Division 3, Quarter 2: $28963.32 [Enter]
Division 3, Quarter 3: $25353.55 [Enter]
Division 3, Quarter 4: $32615.88 [Enter]

The total sales for the company are: $456782.34
```

# 2D ARRAY INITIALIZATION

- Two-dimensional arrays are initialized row-by-row:

```
const int ROWS = 2, COLS = 2;
int exams[ROWS][COLS] = { {84, 78},
                          {92, 97} };
```

| | |
|---|---|
| 84 | 78 |
| 92 | 97 |

- Can omit inner `{ }`, some initial values in a row – array elements without initial values will be set to `0` or `NULL`

# TWO-DIMENSIONAL ARRAY AS PARAMETER, ARGUMENT

- Use array name as argument in function call:

```
getExams(exams, 2);
```

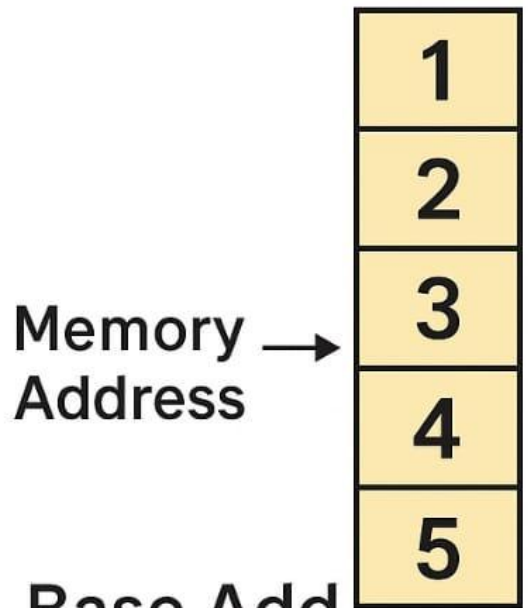
- Use empty `[]` for row, size declarator for column in prototype, header:

```
const int COLS = 2;
// Prototype
void getExams(int [][COLS], int);

// Header
void getExams(int exams[][COLS], int rows)
```

int arr[2][3];

| 1 | 2 | 3 |
|---|---|---|
| 4 | 5 | 6 |




arr[i][j] = Base Add $+(i * 3 + j) \cdot \text{sizeof(int)}$

# EXAMPLE – THE SHOWARRAY FUNCTION FROM PROGRAM

```
30  //*************************************************************
31  // Function Definition for showArray                          *
32  // The first argument is a two-dimensional int array with COLS *
33  // columns. The second argument, rows, specifies the number of *
34  // rows in the array. The function displays the array's contents. *
35  //*************************************************************
36
37  void showArray(int array[][COLS], int rows)
38  {
39     for (int x = 0; x < rows; x++)
40     {
41        for (int y = 0; y < COLS; y++)
42        {
43           cout << setw(4) << array[x][y] << " ";
44        }
45        cout << endl;
46     }
47  }
```

# HOW SHOWARRAY IS CALLED

```
15      int table1[TBL1_ROWS][COLS] = {{1, 2, 3, 4},
16                                     {5, 6, 7, 8},
17                                     {9, 10, 11, 12}};
18      int table2[TBL2_ROWS][COLS] = {{10, 20, 30, 40},
19                                     {50, 60, 70, 80},
20                                     {90, 100, 110, 120},
21                                     {130, 140, 150, 160}};
22
23      cout << "The contents of table1 are:\n";
24      showArray(table1, TBL1_ROWS);
25      cout << "The contents of table2 are:\n";
26      showArray(table2, TBL2_ROWS);
```

# SUMMING ALL THE ELEMENTS IN A TWO-DIMENSIONAL ARRAY

- Given the following definitions:

```
const int NUM_ROWS = 5; // Number of rows
const int NUM_COLS = 5; // Number of columns
int total = 0;          // Accumulator
int numbers[NUM_ROWS][NUM_COLS] =
   {{2, 7, 9, 6, 4},
    {6, 1, 8, 9, 4},
    {4, 3, 7, 2, 9},
    {9, 9, 0, 3, 1},
    {6, 2, 7, 4, 1}};
```

# SUMMING ALL THE ELEMENTS IN A TWO-DIMENSIONAL ARRAY

```
// Sum the array elements.
for (int row = 0; row < NUM_ROWS; row++)
{
   for (int col = 0; col < NUM_COLS; col++)
      total += numbers[row][col];
}

// Display the sum.
cout << "The total is " << total << endl;
```

# SUMMING THE ROWS OF A TWO-DIMENSIONAL ARRAY

- Given the following definitions:

```
const int NUM_STUDENTS = 3;
const int NUM_SCORES = 5;
double total;   // Accumulator
double average; // To hold average scores
double scores[NUM_STUDENTS][NUM_SCORES] =
     {{88, 97, 79, 86, 94},
      {86, 91, 78, 79, 84},
      {82, 73, 77, 82, 89}};
```

# SUMMING THE ROWS OF A TWO-DIMENSIONAL ARRAY

```
// Get each student's average score.
for (int row = 0; row < NUM_STUDENTS; row++)
{
   // Set the accumulator.
   total = 0;
   // Sum a row.
   for (int col = 0; col < NUM_SCORES; col++)
      total += scores[row][col];
   // Get the average
   average = total / NUM_SCORES;
   // Display the average.
   cout << "Score average for student "
        << (row + 1) << " is " << average <<endl;
}
```

# SUMMING THE COLUMNS OF A TWO-DIMENSIONAL ARRAY

- Given the following definitions:

```
const int NUM_STUDENTS = 3;
const int NUM_SCORES = 5;
double total;   // Accumulator
double average; // To hold average scores
double scores[NUM_STUDENTS][NUM_SCORES] =
     {{88, 97, 79, 86, 94},
      {86, 91, 78, 79, 84},
      {82, 73, 77, 82, 89}};
```

# SUMMING THE COLUMNS OF A TWO DIMENSIONAL ARRAY

```
// Get the class average for each score.
for (int col = 0; col < NUM_SCORES; col++)
{
   // Reset the accumulator.
   total = 0;
   // Sum a column
   for (int row = 0; row < NUM_STUDENTS; row++)
      total += scores[row][col];
   // Get the average
   average = total / NUM_STUDENTS;
   // Display the class average.
   cout << "Class average for test " << (col + 1)
        << " is " << average << endl;
}
```

# ARRAYS WITH THREE OR MORE DIMENSIONS

# ARRAYS WITH THREE OR MORE DIMENSIONS

- Can define arrays with any number of dimensions:

```
short rectSolid[2][3][5];
double timeGrid[3][4][3][4];
```

- When used as parameter, specify all but 1st dimension in prototype, heading:

```
void getRectSolid(short [][3][5]);
```

# 1.9.TEMPLATE IN C++

# WHAT ARE TEMPLATES?

- Definition: Templates allow writing generic and reusable code
- Purpose: Write once, use with different data types
- Types:
    - Function Templates
    - Class Templates
- Benefits:
    - Code reusability
    - Type safety
    - Performance (compile-time resolution)

# WHY TEMPLATES?

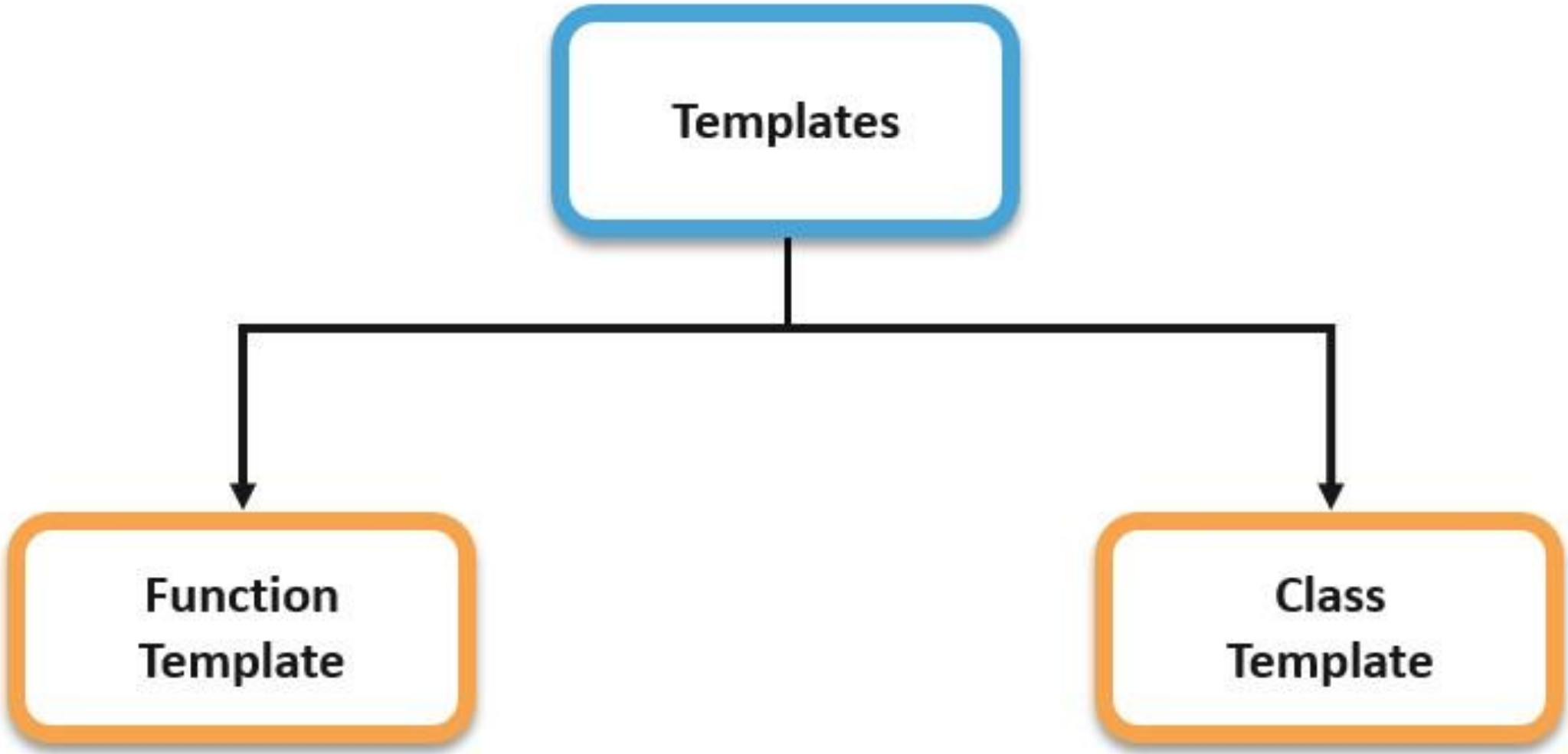
Overloading

```
int max(int a, int b) {
    if (a>= b)
        return a;
    else
        return b;
}
```

```
float max(float a, float b) {
    if (a>= b)
        return a;
    else
        return b;
}
```



Template


Templates
Function Template
Class Template

# 1.9.1. FUNCTION TEMPLATES

- Function templates define a blueprint for functions that enables a function to operate on different data types without rewriting the same logic.
- The syntax for the general form of a template function definition is:

```
template <typename T>

return_type function_name(T parameter) {

  // code

}
```

# 1.9.1. FUNCTION TEMPLATES (cont.)

- Code Example:

```
template <typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}
```

# 1.9.1. FUNCTION TEMPLATES (CONT.)

- Code Example:

```
// Usage
int main() {
    cout << maximum(5, 10) << endl;      // int
    cout << maximum(3.14, 2.71) << endl; // double
    cout << maximum('a', 'z') << endl;   // char
}
```

# 1.9.2. CLASS TEMPLATES

- Similarly, class templates also define a blueprint for creating classes that can work with any data type.

```
template <typename T>

class ClassName {

  // members and methods using T

};
```

# 1.9.2.CLASS TEMPLATES (cont.)

- Code Example 1:



```cpp
template <typename T>
class Box {
  public:
    T value;
    Box(T v) {
      value = v;
    }
    void show() {
      cout << "Value: " << value << "\n";
    }
};

int main() {
  Box<int> intBox(50);
  Box<string> strBox("Hello");

  intBox.show();
  strBox.show();
  return 0;
}
```

# 1.9.2.CLASS TEMPLATES (cont.)

- Code Example 2:



```cpp
template <typename T1, typename T2>
class Pair {
  public:
    T1 first;
    T2 second;

    Pair(T1 a, T2 b) {
      first = a;
      second = b;
    }

    void display() {
      cout << "First: " << first << ", Second: " << second << "\n";
    }
};

int main() {
  Pair<string, int> person("John", 30);
  Pair<int, double> score(51, 9.5);

  person.display();
  score.display();

  return 0;
}
```

# THANKS